Márta Judit Viharos

September 2026

1 Introduction

This document aims to present a short supporting material for the autoregulation and transport modules of first_blood. first_blood is a numerical simulation that solves fundamental fluid mechanics equations for a given model. The large arteries and veins are usually modeled in 1D, while the small arteries, veins, and the capillaries are modeled with 0D lumped models. The abstract structure of the code is presented in Figure 1.

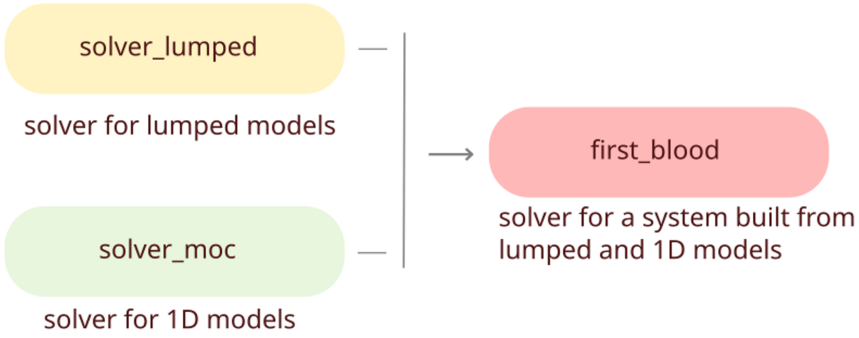


Figure 1: UML diagram of the fundamental and most important classes of first_blood.

solver_lumped is a solver class responsible for the 0D models, while solver_moc is a class handling the 1D models. A model representing the whole vascular system usually consists of many 1D and 0D models, and the first_blood class is responsible of controling all the solver_lumped and solver_moc objects. Each model is in it's own time instant. first_blood advances in time the models one by one with a small timestep, and that is how a simulation propagates in time.

2 Autoregulation

Autoregulation refers to the control mechanisms within the vascular network. In simple terms, more often than not, the arterioles contract and dilate depending on something. And that something can be many things depending on the

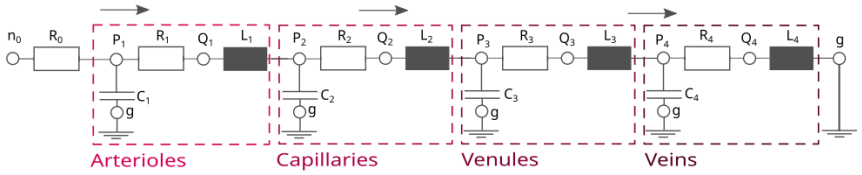


Figure 2: Peripheral model, each compartment modeled with an RLC circuit.

mechanisms itself. Currently the simulation can model the myogenic, metabolic, CO_2 control and the baroreflex. These all effect the arterioles (small arteries) of the vascular network. The arterioles are included in the peripheral models with an RLC circuit, see Figure 2.

The dilation and contraction of the arterioles has a great effect on the resistance (R), and by modifying that the body can control the blood flow to various organs. We can model that by making the arteriolar resistance R_1 dependent on things that autoregulation react to. For example local pressure (myogenic autoregulation). The next part of the documantation will cower how the arteriolar reristances modified in the code.

2.1 Autoregulation in first_blood

The implementation logic can be observed in Figure 3.

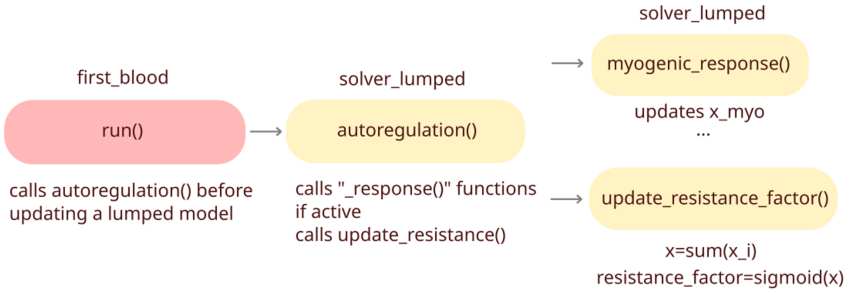


Figure 3: UML diagram of the autoregulation implementation.

As previously mentioned, first_blood is a time dependent numerical simulation. It moves forward in time by selecting the models one by one and propagating them forward with time steps. If a lumped model is to be updated, the autoregulatin() function is called first. This function is responsible for updating the peripheral resistances for a given lumped model.

```
void solver_lumped::autoregulation(double t_act){

    if(t_act<=settling_time){return;}

    bool update = false;

    if(do_myogenic){
        //updates x_myo
        myogenic_response(t_act);
        update = true;
    }

    ...

    if(update){
    update_resistance_factor();
    }
}
```

Figure 4: Code snippet from the autoregulation() function.

The function doesn't do anything before a settlig time (the simulation is transient so we have to wait for settling). The settling_time parameter is a burnt in variable in the solver_lumped class.

Each type of autoregulation process reacts to different things. The myogenic to local pressure, the CO_2 control for local CO_2 content, the metabolic to local tissue O_2 content, etc... So each of these needs to be monitored. One function is written for each and every control mechanism to update the corresponding state variable representing the vasoactive effect.

```
void solver_lumped::myogenic_response(double t_act)
{
    // time step
    double dt = t_act - time.back();
    double p = p_ave->average.back();

    // actuator signal
    x_myo = x_myo + dt / tao * (- x_myo + G_myo * (p - p_ref)/(p_ref - atmospheric_pressure/mmHg_to_Pa));
}
```

Figure 5: Code of the myogenic_response() function.

x_{myo} represents the normed pressure deviation from homeostasis. The low-pass filter removes the high frequency components of the signal. The parameters, like the gain, reference pressure are given in the file of the lumped model. Once all parameters are updated, the update_resistance_factor() function is called (Figure 6).

```
void solver_lumped::update_resistance_factor(){

    double x = x_met + x_myo - x_CO2 + x_bar;
    double sigmoid_scale = (Rmax - Rmin) / (1.0 - Rmin) - 1.0;
    double resistance_factor = Rmin + (Rmax - Rmin) / (1.0 + sigmoid_scale * exp(- sigmoid_gain * x));

    for(int i=0; i<Ridx.size(); i++)
    {
        edges[Ridx[i]]->parameter_factor = resistance_factor;
    }

}
```

Figure 6: Code of the update_resistance_factor() function.

The above function determines and sets the resistance factors of the arterioles. x represents the deviation from homeostasis as the sum of all the x_ variables. These can be added, since they are normed. The sign should be determined based on biological considerations, such as increasing O_2 content causes vasoconstriction (and increase in resistance), while increasing CO_2 results in vasodilation (and a decrease in resistance). Ridx contains the resistances to be modified (R0 and R1 on Figure 2).

Important properties of the autoregulation implementation:

- Various control mechanisms operate independently (E.g.: myogenic response is independent from the CO_2 control). Biologically this is most likely true (although I have no real reference for that). This simplifies things a lot and otherwise the different control mechanisms couldn't be turned on and off and tested independently.
- Individual settings can be adjusted for each peripheral. Various organs likely have different autoregulation responses with different characteristics.
- It's easily extend the simulation with other control mechanisms.

An important note: all x_{id} variables are initialized to 0. If one mechanism is turned off, the corresponding x_{id} parameter never gets updated, and in the update_resistance_factor() function it does not effect the x variable as it adds 0 to the sum.

2.2 Extending the simulation with other control mechanisms

How to add a control mechanism acting on arterioles?

- Determine what the control mechanism reacts to. This can be sg. that we can measure in the simulation (E.g.: local pressure for myogenic simulation) or sg we cannot (E.g.: the baroreflex is really complicated and it is much easier to prescribe the effect of baroreceptor stimuli).
- The main idea is to monitor the cause of the vasoactivity with a dimensionless variable. All are represented with a variable x_{id}, where id denotes the type of autoregulation (E.g.: x_{myo}).

- x_{id} should be defined in such a way that it is 0 at homeostasis. So x_i monitors how strong the vasoactive effect is relative to the nominal value. Like x_{myo} reacts to pressure, so it should be defined as $x_{myo} = \frac{p-p_{ref}}{p_{ref}-p_0}$, where p is the local pressure, p_{ref} is the local pressure corresponding to homeostasis and p_0 is the atmospheric pressure.

- Yeah, it's not that easy. Since the simulation after converging is usually close to a periodic solution, the pressure should be averaged in time for the last heart cycle (with a time period that is dependent on time). An objet of time_average class can do that. So we always need a variable monitorting a vasoactive stuff. For the myogenic response, it is called p_ave (note that these averages should be updated in EVERY TIEMESTEP, don't forget it pls).

- The averaging isn't enough in itself, we also need a low-pass filter, so finally:

$$\tau_{myo}\frac{dx_{myo}}{dt} = -x_{myo} + G_{myo}\left(\frac{\bar{p}-p_{ref}}{p_{ref}-p_0}\right) \tag{1}$$

  Here $\bar{p}$ is the time averaged local pressure of the last cycle, G_{myo} is the gain (larger gain means larger effect) and τ_{myo} is a time constant. τ_{myp} determines how fast the autoregulation responds to stimuli. The equations of most x_{id}-s look similar to this. The only exception is the peripheral baroreflex, since $x_{bar}(t)$ is given in a file before runtime.

- The new x_{id} variable should be updated in a similar function to myogenic_response(). That function should be called in autoregulation() before update_resistance(). And the update_resistance() function should be updated with an x including x_{id}. Other than this the I/O should be written in solver_lumped_io.cpp.

2.3 Autoregulation definition in the model

How can the control mechanisms be given in the files? The CO_2, O_2 and myogenic controls are all given in the lumped model files. An example is presented in Figure 7.

```
data of myogenic control
type, on/off, q_ref, p_ref, tao, G
myogenic, on, 0.0, 103538.69671066948, 8.0, 1.9

data of metabolic control
type, on/off, Ct_ref, tao_met, G_met
metabolic, off, 0.0018049358731558906, 40, 0.9, 0.1, 4

data of CO2 control
type, on/off, Co2_ref, tao_CO2, G_CO2
CO2_response, off, 0.02551527150916738, 8.0, 3.0
```

Figure 7: Example on how to define autoregulation in peripheral models.

For each peripheral model, these control mechanisms can be defined as presented. Each can be turned on or off, and the most important parameters be set, like the gain and time constants. I've previously explained, how all the control mechanisms all effect the same arteriolar resistance. The sigmoidal curve corresponds to all control mechanisms. There are default saturation values of the sigmoidal curve burned in solver_lumped.h. Or it can be given in the file at the metabolic line, if the metabolic control is turned on (Yeah it's terrible I know, I'm sorry). This should definitely be rewritten.

2.4 Peripheral baroreflex

The peripheral baroreflex is a little different. Baroreceptor stimuli can have an effect on peripheral resistances. The process of the phenomenon is quite complicated, and including all the steps in the simulation would be really difficult. For that reason, we kindof only prescribe x_{bar} at the peripherals.

Let me give an example. We want to model the CCC (common carotid compression) test. During this test, the doctor puts pressure on the common carotis artery. This makes the area of the carotis artery smaller, but also puts pressure on the baroreceptors. The baroreceptors "think" that the blood pressure is really high, and signals the brain this, resulting in vasodilation. We can arbitrarily model the carotid compression in the model with an inserted 0D model and a time dependent resistance. And the easiest to model the vasoactive effect of this, is to prescribe a corresponding x_{bar} variable on the peripherals of the brain. One example is presented in Figure 8.

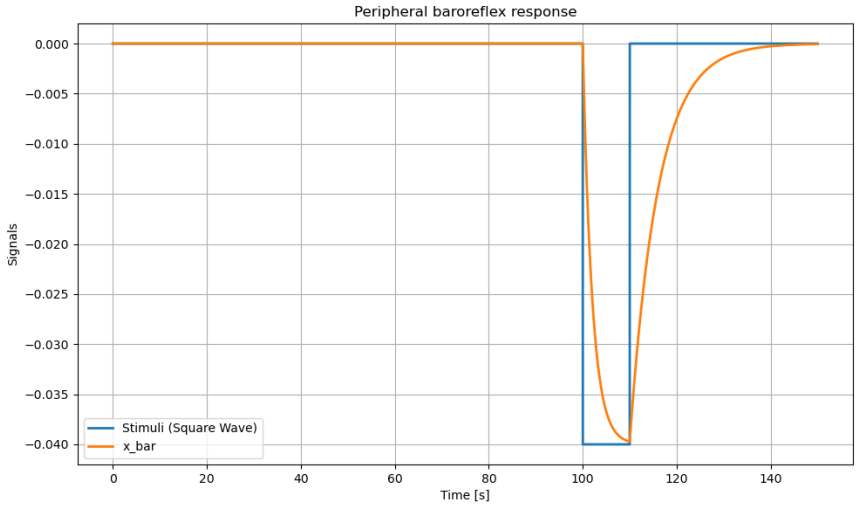


Figure 8: The blue curve represent a how an instant response to the stimuli would look like. But biological systems are not that sharp and do not react that fast, so we get the orange, more realistic curve with a first order lag.

The prescribed x_{bar} variable should be in sync in time with the carotis compression. Like the example on Figure 8 corresponds to carotis compression from 100 to 110 seconds in the simulation time.

The peripheral baroreflex can be given in main.txt, with a line similar to presented in Figure 9.

```
type, on/off, response file, lumped model names
perifbaroreflex, on, carotiscompression, p29, p30, p31, p32
```

Figure 9: Defining the peripheral baroreflex in main.txt. The first is the "perifbaroreflex" keyword, the second is the on/off switch. The third gives the filename of the txt file where the x_{bar} is given as a function of time. The names after that give the peripheral names in which this control is active.

(On GitHub the Abel_A62_63_control model has the peripheral baroreflex.)

3 Transport

Transport refers to monitoring dissolved gas content in blood. At the moment, O_2 and CO_2 represented by their components. The mathematics of these are written in my BSc and MSc thesis, I will not cover any of the math stuff here.

The simulation of transport processes is implemented for both the 1 and 0 dimensional parts of the simulation. The latter is a lot more complicated, so let's start with the first one.

(On GitHub the Bathsheba_CO2_pul model has all kinds of transport implemented. For 1D, peripheral and also pulmonary circulation. I think to use transport in the simulation it is best to study an already built model.)

3.1 Transport in 1D

There are two classes responsible for transport in 1D. Transport_1D handles the edges and Transport_node the nodes. Both of these have one main task, and that is to update various concentration values for edges and nodes respectively. In both classes the update_fi() function does that. For each type of transport variable you have to make a Transport_1D and a Transport_node class.

First_blood updates the velocity, pressure ... parameters of the 1D edges one by one. When an edge is updated, the concentration and transport variables get updated for the 1D neighboring nodes and the edge. If the edge is connected to a lumped model, than that gets updated as well.

The transport processes can be given in the model files. The transports can be set to on or off in main.txt. If it is set to on, the transport is calculated for the 1D models. But the 1D models have to be initialized separately in their own file. Mainly because the arterial and venous blood are really different.

3.2 Transport in 0D

An important difference between the 1 and 0 dimensional transport model, is that the former uses the same edges as the fluid mechanics part, but the latter needs an individual virtual 0D model defined. Because there is no velocity and length scale in 0D.

In the 0D model files, the *virtual 1D model* responsible for the transport can be given as presented in Figure 10. The model consists of various kinds of edges. Currently there are four possibilities, "0Delastance", "0Dcapacitor", "t_diode" and "v1D". Some given parameters need some explanation. The "node start" and "node end" give the names of the start and end nodes of the hydraulic model that we want to use. The virtual one dimension uses the same nodes as the lumped model. The "corresponding edge" refers to the edge in the hydraulic model whose volumetric flow rate is used to calculate the velocity for this virtual 1D edge. (The velocity is calculated as q/A, where q is the volumetric flowrate of the "corresponding edge" and A is the "cross section"). This is different for each kind of edge, and it is easily understandable from the code in solver_lumped_io.cpp.

```
virtual 1D
type, name, node start, node end, length, cross section, division points, is sys. capillary, is pul. capillary, corresponding edge
v1D, arteriole, n1, P2, 0.069705860328302 91, 1.3003287611368157e-05, 10, 0, 0, R1
v1D, capillary, P2, P3, 0.0018333567671206539, 0.0002600565476506 2883, 80, 1, 0, R2
v1D, venulare, P3, P4, 0.051934261774761646, 7.874387758682542e-06, 10, 0, 0, R3

type, name, node start, node end, capacitor
0Dcapacitor, c1, P1, g1, C1
0Dcapacitor, c2, P2, g2, C2
0Dcapacitor, c3, P3, g3, C3
```

Figure 10: An example of a peripheral transport model.

The D0_transport class "builds" an edge and node network for itself. To update the concentrations it updates all nodes and edges based on the previously calculated volumetric flowrate fields. So it is not like the hydraulic model where we use a big matrix and put everything in it, since we can have all necessary information from the vfr field and can update the edges and nodes one by one. (And there is perfect mixing in the nodes for both the 1D and virtual 1D models.)

A few remarks on the transport simulation in 0D:

- The built network should follow the hydraulic model in some way. I mean if the flow branches at a node, there should be as many transport edges as many directions the flow takes. like on Figure 2 at node P_2. From P_2, fluid can go to three directions, therefore three transport edges should join in the same point. The logic is that if blood flow in a direction, we should monitor that. Otherwise the conservation law could not be fulfilled.
- The parameters of the "v1D" edges should be selected *well*. The length should be guessed from non-existing literature and tuned to achieve realistic results. The cross section should be chosen so that the average velocity in the arterioles, capillaries and venules are *realistic*. In my code, I chose 0.02, 0.0005, and 0.033 $\frac{m}{s}$ as the average velocities for the respective compartments. However, feel free to use more appropriate values if you come across a reliable source in the literature. So the cross section can be calculated by:
  - Measuring an average volumetric flowrate of the "corresponding edge" from a precious simulation. For one heart cycle.
  - Dividing that by the desired average velocity from literature.
- So there is a base simulation and we have to use some calculated results from that to model transport for lumped models.
- The four types of edges each have different functionalities and work different. The "0Dcapacitor" is little weird and I could not figure out what the problem is with that.

3.3 Problems with the current transport code design

The classes responsible for transport are *shallow*, meaning that they have little functionality, and the function calls are complicated. A lot of parameters need

to be passed to them. It would make the code cleaner and easier to read, if the Transport_1D and Transport_node classes would be merged into solver_moc. But that would result in a large class with a lot of functionality, and that is also not great. Each class should have one responsibility. Like reading the input model files, or running the simulation, or writing out the results.

Another flaw: the various transport types are handled with a lot of repeated code. This was fine with two or three transport variables, but with eight it makes the code hard to read and prone to errors. This can be avoided with an unordered_map, that maps the transport type to the transport configurations. A part of the code was rewritten with that and it worked well (see in solver_lumped.cpp at the TransportConfig srtucture).

3.4 CO_2 and O_2 transport parameters

There are many transport parameters that are not location specific but should not burnt-in to the simulation since the correct value is not really known. Like solubility values and reaction time constants. These can be set in the model folder in the "CO2_parameters" for CO_2 and in the "O2_parameters" file for O_2 transport. If these files are not found, the simulation will run with default burnt-in values given in the solver_lumped.h file. See the Bathsheba_CO2_pul model as an example on GitHub.